lrdbench: Library Architecture Design and Specifications

1 Core Object Schema and Contracts

This section specifies the core object schema and interface contracts for the benchmarking framework. The purpose of this schema is twofold: first, to provide a mathematically and computationally coherent internal representation of benchmark entities; second, to ensure that all benchmark runs are reproducible, extensible, and auditable. The framework is designed around the principle that each benchmark run must be reconstructible from explicit object metadata, estimator declarations, provenance records, and serialisable experiment manifests.

1.1 Design principles

The core schema is governed by the following principles.

P1. Explicit estimand declaration. Every estimator must declare the quantity it is intended to estimate. The framework must not assume that all methods target the same mathematical object.

P2. Mode-aware evaluation. Truth-based metrics are valid only when a ground-truth target exists. Observational datasets must instead be evaluated through stability, robustness, and consistency criteria.

P3. Failure transparency. Estimator failures, invalid outputs, and warnings must be recorded explicitly rather than silently discarded.

P4. Provenance preservation. Every transformed or estimated time series must retain a full record of its origin, processing history, and benchmark context.

P5. Serialisability. All core objects must admit a lossless serialisation to a machine-readable format such as JSON, YAML, CSV metadata, or Parquet-compatible schema.

P6. Extensibility. New generators, contaminations, estimators, metrics, and report builders must be incorporable through stable contracts rather than through ad hoc code modification.

1.2 Core object schema

Definition 1.1 (Truth specification)**.** *A* truth specification*, denoted by*

$$\mathsf{TruthSpec} = (\texttt{process_family}, \theta, \texttt{target_estimand}, \texttt{target_value}, \texttt{validity_domain}, \texttt{notes}),$$

is the object that records the generative truth associated with a synthetic or otherwise labelled time series.

The fields of TruthSpec are defined as follows.

- `process_family`: the generating family, for example `fGn`, `fBm`, `ARFIMA`, `MRW`, or `fOU`.
- θ: the full set of generating parameters.
- `target_estimand`: the declared target quantity against which an estimator is to be judged, for example a Hurst-type exponent, a long-memory parameter d, or another scaling or regularity parameter.
- `target_value`: the numerical truth corresponding to the declared target estimand.
- `validity_domain`: optional constraints under which the target is considered interpretable or admissible.
- `notes`: optional explanatory text, especially important for processes such as MRW or fOU where a single parameter may not by itself determine the inferential target.

Remark 1.2. *The framework must not reduce truth to a single field* `H`*. A synthetic process may be driven by one parameter while the benchmark target is another. Consequently, truth must be stored as an explicit* estimand-target pair*, not merely as a raw model parameter.*

Definition 1.3 (Series record)**.** *A* series record*, denoted by*

$$\mathsf{SeriesRecord} = (\mathtt{id}, x, t, f_s, \mathtt{source_type}, \mathtt{source_name}, \mathsf{TruthSpec}, \mathcal{H}_{\text{cont}}, \mathcal{H}_{\text{prep}}, \mathcal{A}, \mathcal{M}),$$

is the canonical internal representation of a single time series together with its provenance and metadata.

Its components are:

- `id`: a unique immutable identifier.
- $x = (x_1, \ldots, x_n)$: the observed signal values.
- t: an optional time axis; if omitted, regular sampling is assumed.
- f_s: the sampling frequency or, more generally, sampling metadata.
- `source_type`: one of `synthetic`, `contaminated`, `observational`, or `custom`.
- `source_name`: a human-readable source descriptor.
- TruthSpec: the associated truth specification if available; otherwise `null`.
- $\mathcal{H}_{\text{cont}}$: the contamination history, represented as an ordered list of contamination transformations and parameters.
- $\mathcal{H}_{\text{prep}}$: the preprocessing history, represented as an ordered list of preprocessing operations and parameters.
- $\mathcal{A}$: auxiliary annotations, such as subject identifiers, channel labels, segment information, or biomedical metadata.
- $\mathcal{M}$: provenance metadata, including creation timestamp, parent identifiers, generator version, dataset version, and manifest lineage.

Definition 1.4 (Estimator specification)**.** *An* estimator specification*, denoted by*

$$\begin{aligned}\mathsf{EstimatorSpec} = (&\mathtt{name}, \mathtt{family}, \mathtt{target_estimand}, \mathtt{assumptions}, \mathtt{supports_ci},\\ &\mathtt{supports_diagnostics}, \mathtt{input_requirements}, \mathtt{parameter_schema},\\ &\mathtt{reference_citations}, \mathtt{version})\end{aligned}$$

is the declarative metadata object describing an enrolled estimator.

Its fields are:

- `name`: estimator name.
- `family`: broad methodological family, for example spectral, time-domain, wavelet, geometric, or multifractal.
- `target_estimand`: the quantity the estimator is intended to estimate.
- `assumptions`: explicit assumptions such as stationarity, finite variance, Gaussianity, or regular sampling.
- `supports_ci`: whether the estimator natively provides confidence intervals or standard errors.
- `supports_diagnostics`: whether auxiliary diagnostic outputs are available.
- `input_requirements`: structural requirements on the input signal, for example minimum length, absence of missing values, or sampling assumptions.
- `parameter_schema`: the estimator hyperparameter schema and default values.
- `reference_citations`: methodological references for the estimator.
- `version`: implementation version identifier.

Remark 1.5. *The field* `target_estimand` *is mandatory. No estimator may be benchmarked without declaring the mathematical quantity it purports to estimate.*

Definition 1.6 (Estimate result)**.** *An* estimate result*, denoted by*

$$\begin{aligned}\mathsf{EstimateResult} = (&\mathtt{point}, \mathtt{ci_low}, \mathtt{ci_high}, \mathtt{stderr}, \mathcal{D},\\ &\mathtt{runtime_seconds}, \mathtt{valid}, \mathcal{W}, \mathtt{failure_reason},\\ &\mathtt{estimator_version})\end{aligned}$$

is the canonical return object produced by applying an enrolled estimator to a SeriesRecord*.*

Its fields are:

- `point`: the point estimate.
- `ci_low`, `ci_high`: confidence interval bounds if available.
- `stderr`: standard error if available.
- $\mathcal{D}$: structured diagnostics such as regression fit quality, scale counts, convergence flags, or intermediate summaries.

- `runtime_seconds`: measured runtime.
- `valid`: Boolean validity flag.
- $\mathcal{W}$: warnings generated during estimation.
- `failure_reason`: a structured failure explanation if estimation failed or produced a non-admissible output.
- `estimator_version`: implementation version for provenance.

Definition 1.7 (Benchmark manifest)**.** *A* benchmark manifest*, denoted by*

$$\mathsf{BenchmarkManifest} = (\mathit{name}, \mathit{mode}, \mathcal{G}, \mathcal{C}, \mathcal{S}, \mathcal{E}, \mathcal{Q}, \mathcal{R}, \mathcal{P}, \mathcal{Z}),$$

is the complete declarative specification of a benchmark run.

Its components are:

- `name`: benchmark identifier.
- `mode`: one of `ground_truth`, `stress_test`, or `observational`.
- $\mathcal{G}$: generator or dataset source specification.
- $\mathcal{C}$: contamination specification, possibly empty.
- $\mathcal{S}$: segmentation and preprocessing specification.
- $\mathcal{E}$: the set of enrolled estimator specifications.
- $\mathcal{Q}$: metric and evaluation specification.
- $\mathcal{R}$: report specification.
- $\mathcal{P}$: parallelisation and execution settings.
- $\mathcal{Z}$: random seed policy and reproducibility settings.

Definition 1.8 (Metric bundle)**.** *A* metric bundle*, denoted by*

$$\mathsf{MetricBundle} = (\mathcal{M}_{\text{series}}, \mathcal{M}_{\text{aggregate}}, \mathcal{I}_{\text{strata}}, \mathcal{U}),$$

collects the metrics derived from a completed benchmark run.

Here:

- $\mathcal{M}_{\text{series}}$: per-series metrics, preserving estimator–series pairing.
- $\mathcal{M}_{\text{aggregate}}$: aggregated metrics across replications, conditions, or strata.
- $\mathcal{I}_{\text{strata}}$: index metadata for process family, contamination class, severity level, dataset, segment, estimator family, and other benchmark strata.
- $\mathcal{U}$: uncertainty summaries for benchmark metrics, for example bootstrap intervals on mean absolute error or empirical coverage.

Definition 1.9 (Report bundle)**.** *A* report bundle*, denoted by*

$$\begin{aligned}\mathsf{ReportBundle} = (&\texttt{summary_table}, \texttt{leaderboards}, \texttt{figures},\\ &\texttt{latex_tables}, \texttt{html_report}, \texttt{manifest_copy},\\ &\texttt{result_store_path})\end{aligned}$$

is the complete set of exported artefacts associated with a benchmark run.

1.3 Interface contracts

The framework shall enforce the following contracts.

Contract 1.10 (Generator contract)**.** *A generator is a callable object*

$$\mathsf{Generator} : \mathsf{GeneratorSpec} \longrightarrow \mathsf{SeriesRecord}$$

satisfying the following conditions.

(G1) The returned SeriesRecord *must contain a non-empty signal x.*

(G2) If the source is synthetic, then TruthSpec *must be present.*

(G3) The record must contain sufficient provenance metadata to identify generator family, generating parameters, and implementation version.

(G4) The contamination history $\mathcal{H}_{\text{cont}}$ must be empty upon direct generation.

(G5) The object must be serialisable without loss of benchmark-critical information.

Contract 1.11 (Contamination contract)**.** *A contamination operator is a callable object*

$$\mathsf{Contamination} : \mathsf{SeriesRecord} \longrightarrow \mathsf{SeriesRecord}$$

satisfying the following conditions.

(C1) The output record must preserve parent lineage through provenance metadata.

(C2) The contamination history $\mathcal{H}_{\text{cont}}$ must be appended in order, including the contamination class, parameters, severity label, and implementation version.

(C3) The truth specification may be preserved if the benchmark protocol interprets contamination as acting on an underlying signal with known generating target; otherwise the transformed record must explicitly state how truth is to be interpreted after contamination.

(C4) The operator must not silently alter unrelated metadata fields.

(C5) If the contamination changes sampling, length, or observation structure, these changes must be declared explicitly in the output metadata.

Contract 1.12 (Estimator contract)**.** *An estimator is a callable object*

$$\mathsf{Estimator} : \mathsf{SeriesRecord} \longrightarrow \mathsf{EstimateResult}$$

with accompanying EstimatorSpec*, satisfying the following conditions.*

(E1) The estimator must declare its `target_estimand` *prior to enrolment in the benchmark.*

(E2) The estimator must return a structured EstimateResult*, including a validity flag and explicit failure reason where applicable.*

(E3) The estimator must not suppress numerical failures, convergence failures, or domain violations.

(E4) If confidence intervals or standard errors are produced, the method by which they are constructed must be declared in estimator metadata or diagnostics.

(E5) Runtime must be recorded for each successful or unsuccessful estimation attempt.

Contract 1.13 (Evaluation contract)**.** *An evaluation engine is a callable object*

$$\mathsf{Evaluator} : (\mathsf{BenchmarkManifest}, \{\mathsf{SeriesRecord}\}, \{\mathsf{EstimateResult}\}) \longrightarrow \mathsf{MetricBundle}$$

satisfying the following conditions.

(V1) In `ground_truth` *and* `stress_test` *modes, truth-based metrics may be computed only when* TruthSpec *is available and declared compatible with the estimator target.*

(V2) In `observational` *mode, truth-based metrics such as bias, mean absolute error against truth, and empirical coverage against truth must not be reported.*

(V3) The evaluator must preserve paired estimator–series structure so that paired comparisons remain possible downstream.

(V4) Aggregation over replications, conditions, or strata must be performed through explicit, declared rules.

(V5) The metric bundle must include both per-series and aggregated outputs.

Contract 1.14 (Reporting contract)**.** *A report builder is a callable object*

$$\mathsf{Reporter} : (\mathsf{BenchmarkManifest}, \mathsf{MetricBundle}) \longrightarrow \mathsf{ReportBundle}$$

satisfying the following conditions.

(R1) Every report must include a copy of the benchmark manifest or a cryptographic reference to it.

(R2) Every leaderboard must state the benchmark mode, the aggregation rule, and the metric definition used for ranking.

(R3) Failure counts, invalid estimates, and missing uncertainty outputs must be reported explicitly.

(R4) Human-readable reports and machine-readable result stores must be mutually consistent.

(R5) Exported artefacts must preserve version and provenance metadata.

Contract 1.15 (Reproducibility contract)**.** *A completed benchmark run is* reproducible *if and only if the following items are recorded and exportable:*

(R1) benchmark manifest;

(R2) generator and estimator versions;

(R3) dataset or source version identifiers;

(R4) random seed policy and realised seeds;

(R5) hardware and runtime environment metadata;

(R6) result store and report bundle identifiers.

1.4 Implementation-level invariants

The following invariants should be enforced by the software implementation.

I1. Every SeriesRecord must have a unique immutable identifier.

I2. No estimator output may be represented solely as a scalar; it must be wrapped in EstimateResult.

I3. No benchmark result may be recorded without an associated manifest identifier.

I4. All benchmark modes must be explicitly declared; there shall be no implicit default mode in published runs.

I5. Truth-based and observational evaluation pathways must remain separated in the evaluation engine.

I6. Any missing field that is semantically meaningful must be represented explicitly as `null`, `None`, or an equivalent structured missing value marker rather than through omission.

1.5 Minimal abstract interfaces

For implementation purposes, the following abstract signatures summarise the intended contracts:

```
Generator.generate(spec: GeneratorSpec) -> SeriesRecord

Contamination.apply(record: SeriesRecord) -> SeriesRecord

Estimator.fit(record: SeriesRecord) -> EstimateResult

Evaluator.evaluate(
    manifest: BenchmarkManifest,
    records: list[SeriesRecord],
    results: list[EstimateResult]
) -> MetricBundle

Reporter.build(
    manifest: BenchmarkManifest,
    metrics: MetricBundle
) -> ReportBundle
```

Remark 1.16. *These abstract interfaces are not language-dependent. They may be implemented in Python classes, protocol-based interfaces, functional pipelines, or other equivalent software abstractions, provided that the contracts stated above are preserved.*

2 Metric Schema and Evaluation Protocol

This section defines the benchmark metric schema, the admissibility of metrics by benchmark mode, the aggregation rules, and the evaluation protocol governing the computation of estimator performance summaries. The central principle is that benchmark metrics must be *mode-aware*: truth-based metrics are admissible only when a compatible ground-truth target exists, whereas observational datasets require stability- and robustness-based evaluation rather than accuracy against an unavailable truth.

2.1 Benchmark modes

The framework supports three benchmark modes.

M1. Ground-truth mode (`ground_truth`): canonical synthetic time series with an explicitly declared TruthSpec.

M2. Stress-test mode (`stress_test`): synthetic time series with known truth subjected to controlled contamination operators.

M3. Observational mode (`observational`): real-world or custom data for which no benchmark ground truth is available.

Remark 2.1. *The evaluation engine must treat these modes as epistemically distinct. In particular, no metric that depends on the unknown true value of the target estimand may be reported in observational mode.*

2.2 Metric specification schema

Definition 2.2 (Metric specification)**.** *A* metric specification*, denoted by*

$$\begin{aligned}\mathsf{MetricSpec} = (&\mathit{name}, \mathit{symbol}, \mathit{requires_truth},\\ &\mathit{admissible_modes}, \mathit{aggregation_rule},\\ &\mathit{optimisation_direction}, \mathit{unit},\\ &\mathit{null_policy})\end{aligned}$$

is the declarative object describing a benchmark metric.

Its fields are:

- `name`: metric identifier.
- `symbol`: mathematical symbol used in equations and reports.
- `requires_truth`: Boolean flag indicating whether the metric requires a compatible TruthSpec.
- `admissible_modes`: subset of {`ground_truth`, `stress_test`, `observational`}.
- `aggregation_rule`: declared rule for aggregation over replications or strata.
- `optimisation_direction`: one of `minimise`, `maximise`.
- `unit`: metric unit where applicable.
- `null_policy`: rule for handling missing metric values.

2.3 Notation

Let $i \in \{1, \ldots, N\}$ index time series records, and let $e \in \{1, \ldots, E\}$ index estimators. When applicable, let $y_i^\star$ denote the true benchmark target recorded in the corresponding TruthSpec. Let $\hat{y}_{ie}$ denote the point estimate returned by estimator e on series i. If uncertainty is available, let

$$I_{ie}^{(1-\alpha)} = [L_{ie}^{(\alpha)}, U_{ie}^{(\alpha)}]$$

denote a nominal $(1-\alpha)$-level confidence interval. Let T_{ie} denote runtime in seconds, and let $V_{ie} \in \{0, 1\}$ denote the validity indicator, where $V_{ie} = 1$ means that the estimator returned an admissible output.

When analysing stress-test mode, let i° denote the clean progenitor of contaminated record i, so that contaminated and uncontaminated series remain paired through provenance.

When analysing observational mode, let $w \in \{1, \ldots, W_i\}$ index windows, segments, resamples, or perturbation variants associated with record i.

2.4 Truth-based per-series metrics

The following metrics are admissible only when a compatible ground-truth target exists.

Definition 2.3 (Point error)**.** *The signed point error for series i and estimator e is*

$$\varepsilon_{ie} = \hat{y}_{ie} - y_i^\star.$$

Definition 2.4 (Absolute error)**.** *The absolute error is*

$$a_{ie} = |\hat{y}_{ie} - y_i^\star|.$$

Definition 2.5 (Squared error)**.** *The squared error is*

$$s_{ie} = (\hat{y}_{ie} - y_i^\star)^2.$$

Definition 2.6 (Confidence interval coverage)**.** *If a confidence interval is available, the empirical coverage indicator is*

$$c_{ie}^{(\alpha)} = \mathbf{1}\left\{L_{ie}^{(\alpha)} \le y_i^\star \le U_{ie}^{(\alpha)}\right\}.$$

Definition 2.7 (Confidence interval width)**.** *If a confidence interval is available, its width is*

$$w_{ie}^{(\alpha)} = U_{ie}^{(\alpha)} - L_{ie}^{(\alpha)}.$$

Definition 2.8 (Failure indicator)**.** *The failure indicator is*

$$f_{ie} = 1 - V_{ie}.$$

Remark 2.9. *Metrics ε_{ie}, a_{ie}, s_{ie}, and $c_{ie}^{(\alpha)}$ are undefined in observational mode and must not be imputed.*

2.5 Aggregate truth-based metrics

Let $\mathcal{I}_g$ denote the set of record indices belonging to stratum g, where a stratum may be defined by process family, target value, signal length, contamination class, or other declared partition.

Definition 2.10 (Bias)**.** *The stratum-wise bias of estimator e on stratum g is*

$$\mathrm{Bias}_{e,g} = \frac{1}{|\mathcal{I}_g|} \sum_{i \in \mathcal{I}_g} \varepsilon_{ie}.$$

Definition 2.11 (Mean absolute error)**.** *The stratum-wise mean absolute error is*

$$\mathrm{MAE}_{e,g} = \frac{1}{|\mathcal{I}_g|} \sum_{i \in \mathcal{I}_g} a_{ie}.$$

Definition 2.12 (Root mean squared error)**.** *The stratum-wise root mean squared error is*

$$\mathrm{RMSE}_{e,g} = \left(\frac{1}{|\mathcal{I}_g|} \sum_{i \in \mathcal{I}_g} s_{ie} \right)^{1/2}.$$

Definition 2.13 (Empirical coverage)**.** *For nominal level $1-\alpha$, the empirical coverage of estimator e on stratum g is*

$$\mathrm{Cov}_{e,g}^{(\alpha)} = \frac{1}{|\mathcal{I}_g^{(\alpha)}|} \sum_{i \in \mathcal{I}_g^{(\alpha)}} c_{ie}^{(\alpha)},$$

where $\mathcal{I}_g^{(\alpha)} \subseteq \mathcal{I}_g$ is the subset for which confidence intervals are available.

Definition 2.14 (Coverage error)**.** *The coverage error is*

$$\mathrm{CErr}_{e,g}^{(\alpha)} = \left| \mathrm{Cov}_{e,g}^{(\alpha)} - (1-\alpha) \right|.$$

Definition 2.15 (Mean interval width)**.** *The mean confidence interval width is*

$$\mathrm{MW}_{e,g}^{(\alpha)} = \frac{1}{|\mathcal{I}_g^{(\alpha)}|} \sum_{i \in \mathcal{I}_g^{(\alpha)}} w_{ie}^{(\alpha)}.$$

Definition 2.16 (Validity rate)**.** *The validity rate is*

$$\mathrm{VR}_{e,g} = \frac{1}{|\mathcal{I}_g|} \sum_{i \in \mathcal{I}_g} V_{ie}.$$

Definition 2.17 (Mean runtime)**.** *The mean runtime is*

$$\mathrm{RT}_{e,g} = \frac{1}{|\mathcal{I}_g|} \sum_{i \in \mathcal{I}_g} T_{ie}.$$

2.6 Stress-test metrics

Stress-test mode is designed to measure performance degradation under controlled contamination. The key design principle is that every contaminated series must remain linked to a clean progenitor through provenance.

Let i° denote the clean progenitor of contaminated record i. Let $\hat{y}_{i^\circ e}$ denote the estimate on the clean counterpart and $\hat{y}_{ie}$ the estimate on the contaminated record.

Definition 2.18 (Estimate drift)**.** *The contamination-induced estimate drift is*

$$\Delta_{ie} = \hat{y}_{ie} - \hat{y}_{i^\circ e}.$$

Its absolute version is

$$|\Delta_{ie}| = |\hat{y}_{ie} - \hat{y}_{i^\circ e}|.$$

Definition 2.19 (Absolute error inflation)**.** *Let $a_{i^\circ e}$ and a_{ie} denote the clean and contaminated absolute errors. The absolute error inflation is*

$$\mathrm{AEI}_{ie} = a_{ie} - a_{i^\circ e}.$$

Definition 2.20 (Relative degradation ratio)**.** *For a stratum g, the relative degradation ratio is*

$$\mathrm{RDR}_{e,g} = \frac{\mathrm{MAE}_{e,g}^{\mathrm{cont}}}{\mathrm{MAE}_{e,g}^{\mathrm{clean}} + \delta},$$

where $\delta > 0$ is a small numerical stabiliser.

Definition 2.21 (Coverage collapse)**.** *For nominal level $1-\alpha$, the coverage collapse under contamination is*

$$\mathrm{CC}_{e,g}^{(\alpha)} = \mathrm{Cov}_{e,g}^{\mathrm{clean},(\alpha)} - \mathrm{Cov}_{e,g}^{\mathrm{cont},(\alpha)}.$$

Definition 2.22 (Validity collapse)**.** *The validity collapse under contamination is*

$$\mathrm{VC}_{e,g} = \mathrm{VR}_{e,g}^{\mathrm{clean}} - \mathrm{VR}_{e,g}^{\mathrm{cont}}.$$

Definition 2.23 (Robustness score)**.** *A generic robustness score for estimator e on stratum g may be defined through a weighted rank aggregation of degradation-oriented component metrics. In its simplest form,*

$$\mathrm{RobScore}_{e,g} = \sum_{k=1}^{K} \omega_k \, \mathrm{rank}_g\big(m_{e,g}^{(k)}\big), \qquad \sum_{k=1}^{K} \omega_k = 1,$$

where $m_{e,g}^{(k)}$ denotes a declared stress-test metric such as RDR, CC, VC, *or mean absolute drift, and lower scores indicate stronger robustness.*

Remark 2.24. *The framework should begin with rank-based composite scoring rather than an opaque nonlinear scalar objective. Rank aggregation is easier to interpret, easier to audit, and less brittle under heterogeneous metric scales.*

2.7 Observational metrics

Observational mode evaluates estimator behaviour in the absence of benchmark truth. Let $\hat{y}_{iwe}$ denote the estimate produced by estimator e on window, perturbation, or resample w associated with record i.

Definition 2.25 (Within-record instability)**.** *The within-record instability of estimator e on record i is*

$$\mathrm{Instab}_{ie} = \mathrm{SD}(\hat{y}_{i1e}, \ldots, \hat{y}_{iW_ie}),$$

or, in a more robust variant,

$$\mathrm{Instab}_{ie}^{\mathrm{MAD}} = \mathrm{MAD}(\hat{y}_{i1e}, \ldots, \hat{y}_{iW_ie}).$$

Definition 2.26 (Preprocessing sensitivity)**.** *Suppose $p \in \{1, \ldots, P_i\}$ indexes preprocessing variants applied to the same underlying record i. Then the preprocessing sensitivity is*

$$\mathrm{PSens}_{ie} = \frac{1}{P_i(P_i-1)} \sum_{\substack{p,q=1 \\ p\neq q}}^{P_i} |\hat{y}_{ipe} - \hat{y}_{iqe}|.$$

Definition 2.27 (Bootstrap variability)**.** *If block bootstrap or other resampling is applied, and $b \in \{1, \ldots, B_i\}$ indexes resampled realisations, then the bootstrap variability is*

$$\mathrm{BVar}_{ie} = \mathrm{SD}\Big(\hat{y}_{i1e}^{\mathrm{boot}}, \ldots, \hat{y}_{iB_ie}^{\mathrm{boot}}\Big).$$

Definition 2.28 (Observational failure rate)**.** *The observational failure rate is*

$$\mathrm{OFR}_{e,g} = \frac{1}{|\mathcal{I}_g|} \sum_{i\in\mathcal{I}_g} \frac{1}{W_i} \sum_{w=1}^{W_i} (1 - V_{iwe}),$$

where V_{iwe} is the validity indicator on window, resample, or perturbation w.

Definition 2.29 (Observational stability score)**.** *A stability score for estimator e on stratum g may be defined through a weighted rank aggregation of observational metrics such as instability, preprocessing sensitivity, bootstrap variability, and failure rate:*

$$\mathrm{StabScore}_{e,g} = \sum_{k=1}^{K} \eta_k \,\mathrm{rank}_g\big(\tilde{m}_{e,g}^{(k)}\big), \qquad \sum_{k=1}^{K} \eta_k = 1.$$

2.8 Metric admissibility by mode

The admissibility of benchmark metrics is summarised by the following rules.

Metric class	Ground-truth	Stress-test	Observational
Bias / MAE / RMSE	admissible	admissible	inadmissible
Coverage / coverage error	admissible	admissible	inadmissible
CI width	admissible	admissible	admissible if available
Validity rate	admissible	admissible	admissible
Estimate drift	inadmissible	admissible	admissible only as perturbation drift
Runtime / efficiency	admissible	admissible	admissible
Instability / sensitivity	optional	optional	admissible

Remark 2.30. *In observational mode, interval width may still be reported as an internal uncertainty descriptor if the estimator or resampling scheme provides it. However, it must not be interpreted as empirical coverage against truth.*

2.9 Aggregation protocol

The benchmark framework must support aggregation at multiple levels:

A1. Per-series level: raw estimator outputs and per-series metrics.

A2. Stratum level: aggregation over declared strata such as process family, target value, length, contamination class, severity, dataset, or subject.

A3. Global benchmark level: balanced aggregation over strata.

Definition 2.31 (Balanced global aggregation)**.** *Let $\mathcal{G}$ denote the declared set of benchmark strata. For a stratum-wise metric $m_{e,g}$, the balanced global aggregate is*

$$\overline{m}_e = \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} m_{e,g}.$$

Remark 2.32. *Balanced aggregation is preferable to naive pooling when the benchmark is intended to compare performance across heterogeneous strata. Otherwise, a few large strata may dominate the overall metric.*

2.10 Uncertainty quantification for benchmark metrics

Benchmark-level metrics should themselves be reported with uncertainty whenever feasible.

Definition 2.33 (Bootstrap interval for benchmark metrics)**.** *Let $m_{e,g}$ be a stratum-wise summary metric for estimator e. A bootstrap confidence interval for $m_{e,g}$ is obtained by resampling records within stratum g with replacement and recomputing the summary metric across bootstrap replicates.*

Definition 2.34 (Paired bootstrap for estimator differences)**.** *Suppose e_1 and e_2 are applied to the same set of records in stratum g. Let $d_i = m_{ie_1} - m_{ie_2}$ denote the paired per-record difference. A paired bootstrap interval for the estimator difference is obtained by resampling the record indices i with replacement and recomputing the mean or median of d_i.*

Remark 2.35. *Paired resampling is essential whenever estimators are compared on the same underlying set of time series. The benchmark framework must preserve this pairing in the stored result structure.*

2.11 Leaderboard protocol

Leaderboards must be benchmark-mode specific.

L1. Accuracy leaderboard for ground-truth mode.

L2. Robustness leaderboard for stress-test mode.

L3. Stability leaderboard for observational mode.

Definition 2.36 (Leaderboard specification)**.** *A leaderboard specification is*

$$\mathsf{LeaderboardSpec} = (\texttt{mode}, \texttt{component_metrics}, \texttt{weights}, \texttt{ranking_rule}, \texttt{tie_break_rule}).$$

Definition 2.37 (Weighted rank aggregation)**.** *Let $m_e^{(1)}, \ldots, m_e^{(K)}$ denote the chosen aggregate component metrics for estimator e. The weighted rank aggregation score is*

$$\mathrm{RankScore}_e = \sum_{k=1}^{K} \lambda_k \, \mathrm{rank}\Big(m_e^{(k)}\Big), \qquad \sum_{k=1}^{K} \lambda_k = 1.$$

Lower scores correspond to stronger leaderboard performance.

Remark 2.38. *The benchmark reports should always publish the raw component metrics alongside any composite rank score. Composite scores are convenient summaries, not replacements for scientific interpretation.*

2.12 Evaluation protocol by mode

Ground-truth protocol. For each declared synthetic stratum:

(G1) generate replicated synthetic records;

(G2) apply each estimator to each record;

(G3) compute per-series truth-based metrics;

(G4) aggregate within strata;

(G5) compute global balanced summaries and leaderboard rankings.

Stress-test protocol. For each clean synthetic record:

(S1) generate or identify a clean progenitor record;

(S2) apply one or more contamination operators at declared severity levels;

(S3) apply each estimator to both clean and contaminated records;

(S4) compute truth-based metrics and contamination-induced degradation metrics;

(S5) aggregate within contamination strata and severity levels;

(S6) compute robustness leaderboards and degradation plots.

Observational protocol. For each observational record:

(O1) segment, resample, or perturb the record according to the declared protocol;

(O2) apply each estimator to each variant;

(O3) compute instability, sensitivity, variability, and failure metrics;

(O4) aggregate within subjects, datasets, channels, or other observational strata;

(O5) compute stability leaderboards and report sensitivity analyses.

2.13 Reporting requirements for evaluation outputs

Every benchmark report must state the following explicitly:

Q1. benchmark mode;

Q2. estimator set and declared target estimands;

Q3. stratum definitions;

Q4. per-metric admissibility conditions;

Q5. aggregation rule;

Q6. uncertainty quantification method;

Q7. leaderboard construction rule;

Q8. failure and missing-data handling policy.

Remark 2.39. *This reporting discipline is essential because two benchmark summaries that use the same estimators but different aggregation rules, missing-value policies, or stratum balancing schemes are not directly comparable.*

3 Reporting Schema and Artefact Protocol

This section specifies the reporting schema, export artefacts, and report-generation protocol for the benchmarking framework. The objective is to ensure that every benchmark run produces a report that is simultaneously human-readable, machine-auditable, reproducible, and suitable for downstream scientific communication.

3.1 Reporting principles

The reporting layer is governed by the following principles.

RP1. Manifest traceability. Every report must be linked unambiguously to the BenchmarkManifest from which it was generated.

RP2. Metric transparency. Every table, leaderboard, and figure must state or reference the metric definitions and aggregation rules used to produce it.

RP3. Failure visibility. Failures, invalid estimates, unavailable uncertainty outputs, and other non-ideal outcomes must be displayed explicitly.

RP4. Mode-specific interpretation. Report content must respect the benchmark mode. In particular, observational reports must not present truth-based quantities as if they were available.

RP5. Machine readability. All report-level outputs must be backed by serialised machine-readable files.

RP6. Publication readiness. The reporting layer should be able to emit publication-ready tables and figures without requiring manual post hoc reconstruction of benchmark results.

3.2 Report specification schema

Definition 3.1 (Report specification)**.** *A* report specification*, denoted by*

$$\mathsf{ReportSpec} = (\mathit{formats}, \mathit{leaderboards}, \mathit{figure_set}, \mathit{table_set}, \mathit{include_raw_exports}, \mathit{include_provenance}, \mathit{in}$$

is the declarative object that governs report generation for a benchmark run.

Its components are:

- `formats`: output formats such as `html`, `markdown`, `latex`, `csv`, `json`, and `parquet`.
- `leaderboards`: list of LeaderboardSpec objects to be rendered.
- `figure_set`: declared figures to generate.
- `table_set`: declared tables to generate.
- `include_raw_exports`: Boolean flag for raw metric and result exports.
- `include_provenance`: Boolean flag for manifest and provenance inclusion.
- `include_environment`: Boolean flag for runtime environment metadata.
- `export_root`: root directory or storage target for exported artefacts.
- `naming_policy`: rule for naming folders and files.
- `compression_policy`: optional compression rule for large artefacts.

3.3 Result-store schema

The reporting layer should not merely render views of results. It must also persist the underlying benchmark outputs in a structured result store.

Definition 3.2 (Result store)**.** *A* result store*, denoted by*

$$\mathsf{ResultStore} = (\mathcal{R}_{\text{runs}}, \mathcal{R}_{\text{records}}, \mathcal{R}_{\text{estimates}}, \mathcal{R}_{\text{metrics}}, \mathcal{R}_{\text{leaderboards}}, \mathcal{R}_{\text{artefacts}}),$$

is the complete machine-readable collection of benchmark outputs associated with one or more benchmark runs.

The constituent relations are:

- $\mathcal{R}_{\text{runs}}$: run-level metadata, including manifest identifiers, benchmark mode, software version, timestamps, and environment information.
- $\mathcal{R}_{\text{records}}$: series-level metadata, including source type, stratum membership, truth metadata where applicable, contamination history, preprocessing history, and provenance lineage.
- $\mathcal{R}_{\text{estimates}}$: raw estimator outputs, including point estimates, uncertainty outputs, diagnostics, runtime, validity flags, and failure reasons.
- $\mathcal{R}_{\text{metrics}}$: per-series and aggregated benchmark metrics.

- $\mathcal{R}_{\text{leaderboards}}$: leaderboard rows, including ranking positions, component metrics, weights, and tie-break outcomes.
- $\mathcal{R}_{\text{artefacts}}$: exported artefact metadata, including file paths, content hashes, and format descriptors.

Remark 3.3. *The result store should be designed so that every human-readable report can be regenerated from it without re-running the estimators.*

3.4 Artefact schema

Definition 3.4 (Artefact record)**.** *An* artefact record *is a tuple*

$$\mathsf{ArtefactRecord} = (\mathtt{artefact_id}, \mathtt{run_id}, \mathtt{artefact_type}, \mathtt{format}, \mathtt{path}, \mathtt{hash}, \mathtt{created_at}, \mathtt{depends_on}),$$

describing a single exported file or directory produced by the reporting layer.

Typical values of `artefact_type` include:

- `html_report`
- `markdown_report`
- `latex_table`
- `figure`
- `manifest_copy`
- `metric_export`
- `leaderboard_export`
- `environment_snapshot`

3.5 Mandatory report components

Every benchmark report, regardless of mode, must contain the following components.

RC1. Run summary. Benchmark name, run identifier, timestamp, benchmark mode, software version, and manifest reference.

RC2. Estimator registry summary. List of enrolled estimators together with their declared target estimands.

RC3. Benchmark strata summary. Description of process families, signal lengths, contamination classes, severity levels, datasets, or other declared strata.

RC4. Metric and leaderboard definitions. Explicit statement of the metrics used in scoring and ranking, including aggregation and weighting rules.

RC5. Failure summary. Counts and rates of invalid estimates, failed runs, missing uncertainty outputs, and warnings.

RC6. Provenance appendix. Manifest copy or manifest digest, seed policy, environment information, and export paths.

3.6 Mode-specific reporting requirements

The report content must be specialised by benchmark mode.

Ground-truth reports. A ground-truth report must include:

(GT1) accuracy leaderboard;

(GT2) per-stratum bias, MAE, and RMSE summaries;

(GT3) coverage and interval-width summaries where available;

(GT4) runtime and validity summaries;

(GT5) per-family or per-length breakdown tables;

(GT6) failure analysis.

Stress-test reports. A stress-test report must include:

(ST1) robustness leaderboard;

(ST2) degradation summaries relative to clean progenitors;

(ST3) contamination-specific drift analyses;

(ST4) coverage collapse and validity collapse summaries;

(ST5) per-severity breakdown tables;

(ST6) failure analysis.

Observational reports. An observational report must include:

(OB1) stability leaderboard;

(OB2) within-record instability summaries;

(OB3) preprocessing sensitivity summaries;

(OB4) resampling or bootstrap variability summaries where applicable;

(OB5) failure analysis;

(OB6) explicit statement that no truth-based accuracy metrics are reported.

3.7 Table schema

The reporting layer should generate both machine-readable tables and publication-ready tables.

Definition 3.5 (Canonical table classes)**.** *The reporting protocol defines the following canonical table classes.*

(T1) ***Run summary table:*** *one row per benchmark run.*

(T2) ***Estimator metadata table:*** *one row per estimator.*

(T3) ***Per-stratum metric table:*** *one row per estimator–stratum pair.*

(T4) ***Leaderboard table:*** *one row per estimator, including component scores and overall rank.*

(T5) ***Failure table:*** *one row per estimator or estimator–stratum pair, including failure counts and invalidity rates.*

(T6) ***Artefact index table:*** *one row per exported artefact.*

Remark 3.6. *Publication-ready tables should be derived systematically from canonical metric tables rather than created as bespoke summaries. This prevents drift between human-readable summaries and machine-readable exports.*

3.8 Figure schema

The report protocol should support a declared figure set. Typical figure classes include:

(F1) **Leaderboard bar plots.**

(F2) **Per-stratum heatmaps.**

(F3) **Coverage calibration plots.**

(F4) **Degradation curves over contamination severity.**

(F5) **Estimate distribution plots** (e.g. boxplots or violin plots).

(F6) **Runtime versus accuracy plots.**

(F7) **Observational instability and sensitivity plots.**

Definition 3.7 (Figure record)**.** *A figure record must include:*

$$(\texttt{figure_id}, \texttt{run_id}, \texttt{title}, \texttt{caption}, \texttt{source_table}, \texttt{path}, \texttt{format}).$$

Remark 3.8. *Every figure should be traceable to the exact table or metric source from which it was generated. Figures must not be treated as free-standing graphics detached from the underlying result store.*

3.9 Directory and naming protocol

A benchmark run should export artefacts to a deterministic directory structure. A recommended layout is:

```
reports/
  <run_id>/
    manifest/
      benchmark_manifest.yaml
      environment.json
    raw/
      records.parquet
      estimates.parquet
      metrics.parquet
      leaderboards.parquet
    tables/
      run_summary.csv
      estimator_metadata.csv
      per_stratum_metrics.csv
      leaderboard.csv
      failures.csv
    figures/
      leaderboard.png
      degradation_curve.png
      calibration_plot.png
    latex/
      leaderboard.tex
      summary_table.tex
    html/
      report.html
    markdown/
      report.md
    artefacts/
      artefact_index.csv
```

Definition 3.9 (Naming policy)**.** *A naming policy must assign deterministic names to exported artefacts as a function of*

$$(\texttt{run_id}, \texttt{artefact_type}, \texttt{benchmark_mode}, \texttt{stratum_tag}, \texttt{format}).$$

3.10 Provenance and environment exports

Every benchmark report must include provenance metadata sufficient to reproduce or audit the run.

Definition 3.10 (Environment snapshot)**.** *An environment snapshot is a structured export containing at least:*

(E1) software package versions;

(E2) operating system information;

(E3) hardware information where available;

(E4) random seed policy;

(E5) execution timestamp;

(E6) source code version identifier or git commit hash where available.

Contract 3.11 (Report provenance contract)**.** *Every* ReportBundle *must include either:*

(P1) a full serialised copy of the BenchmarkManifest*, or*

(P2) a stable reference and cryptographic digest that uniquely identifies the manifest used.

3.11 Consistency constraints

The following consistency constraints must be enforced by the reporting layer.

CC1. Every leaderboard row must correspond to a valid aggregate metric record in the result store.

CC2. Every rendered table must be reproducible from machine-readable exports.

CC3. Every figure must reference at least one source table or source metric collection.

CC4. Every failure count reported in a summary must reconcile with the estimate-level records.

CC5. Every report must state the benchmark mode and must not display inadmissible metric classes for that mode.

3.12 Minimal report-generation interfaces

For implementation purposes, the following abstract interfaces summarise the intended reporting behaviour.

```
Reporter.build(
    manifest: BenchmarkManifest,
    metrics: MetricBundle,
    report_spec: ReportSpec
) -> ReportBundle

ResultStore.write(
    run_metadata,
    records,
    estimates,
    metrics,
    leaderboards,
    artefacts
) -> path

ArtefactIndex.register(
    artefact_record: ArtefactRecord
) -> None
```

3.13 Interpretation safeguards

The report generator should enforce the following interpretive safeguards.

IS1. Observational reports must display an explicit notice that truth-based accuracy and empirical coverage against truth are not available.

IS2. Stress-test reports must distinguish clean-reference metrics from contamination-induced degradation metrics.

IS3. Composite leaderboard scores must always be accompanied by the raw component metrics from which they were computed.

IS4. Missing uncertainty outputs must not be conflated with narrow uncertainty.

IS5. Failure suppression is forbidden: missing, invalid, or crashed estimator outputs must appear in the report and in the result store.

3.14 Recommended report templates

For practical use, the framework should support at least the following templates:

(RPT1) **Full benchmark report:** detailed HTML or Markdown report with figures, tables, manifest appendix, and result-store links.

(RPT2) **Publication summary report:** concise table- and figure-oriented export suitable for later inclusion in a paper.

(RPT3) **Audit report:** manifest-forward report emphasising reproducibility, provenance, and consistency checks.

Remark 3.12. *These templates are not different scientific products; they are different renderings of the same underlying result store and provenance record.*

4 Python-Facing Abstract Class and Interface Design

This section presents the intended Python-facing abstract interfaces for the benchmarking framework. The purpose of these interfaces is to translate the formal object schema and contracts defined in Sections 1–3 into implementation-ready software abstractions. The design separates immutable data records from pluggable computational components.

4.1 Design orientation

The Python-facing architecture follows three principles.

PI1. Data records are explicit. Core schema objects are represented as serialisable data containers.

PI2. Computational components are pluggable. Generators, contaminations, estimators, evaluators, and reporters are exposed through stable interfaces.

PI3. Orchestration is separate from computation. The benchmark runner coordinates the workflow but does not itself implement estimation, evaluation, or reporting logic.

4.2 Core imports and type aliases

```
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum
from pathlib import Path
from typing import Any, Mapping, Sequence, Protocol, runtime_checkable
```

4.3 Enumerations

```
class BenchmarkMode(str, Enum):
    GROUND_TRUTH = "ground_truth"
    STRESS_TEST = "stress_test"
    OBSERVATIONAL = "observational"


class SourceType(str, Enum):
    SYNTHETIC = "synthetic"
    CONTAMINATED = "contaminated"
    OBSERVATIONAL = "observational"
    CUSTOM = "custom"


class OptimisationDirection(str, Enum):
    MINIMISE = "minimise"
    MAXIMISE = "maximise"
```

4.4 Core data records

```
@dataclass(frozen=True)
class TruthSpec:
    process_family: str
    generating_params: Mapping[str, Any]
    target_estimand: str
    target_value: float | None
    validity_domain: Mapping[str, Any] = field(default_factory=dict)
    notes: str | None = None


@dataclass(frozen=True)
class TransformationRecord:
    name: str
    family: str
    params: Mapping[str, Any]
    severity: str | None = None
    version: str | None = None
    parent_id: str | None = None
    notes: str | None = None
```

```python
@dataclass(frozen=True)
class ProvenanceRecord:
    record_id: str
    parent_id: str | None
    manifest_id: str | None
    created_at: str
    source_version: str | None = None
    software_version: str | None = None
    git_commit: str | None = None
    seed: int | None = None


@dataclass(frozen=True)
class SeriesRecord:
    record_id: str
    values: Sequence[float]
    time_axis: Sequence[float] | None
    sampling_rate: float | None
    source_type: SourceType
    source_name: str
    truth: TruthSpec | None = None
    contamination_history: tuple[TransformationRecord, ...] = ()
    preprocessing_history: tuple[TransformationRecord, ...] = ()
    annotations: Mapping[str, Any] = field(default_factory=dict)
    provenance: ProvenanceRecord | None = None


@dataclass(frozen=True)
class EstimatorSpec:
    name: str
    family: str
    target_estimand: str
    assumptions: tuple[str, ...]
    supports_ci: bool
    supports_diagnostics: bool
    input_requirements: Mapping[str, Any] = field(default_factory=dict)
    parameter_schema: Mapping[str, Any] = field(default_factory=dict)
    reference_citations: tuple[str, ...] = ()
    version: str | None = None


@dataclass(frozen=True)
class EstimateResult:
    record_id: str
    estimator_name: str
    point: float | None
    ci_low: float | None = None
    ci_high: float | None = None
    stderr: float | None = None
    diagnostics: Mapping[str, Any] = field(default_factory=dict)
    runtime_seconds: float | None = None
    valid: bool = True
    warnings: tuple[str, ...] = ()
```

```
    failure_reason: str | None = None
    estimator_version: str | None = None


@dataclass(frozen=True)
class MetricSpec:
    name: str
    symbol: str
    requires_truth: bool
    admissible_modes: tuple[BenchmarkMode, ...]
    aggregation_rule: str
    optimisation_direction: OptimisationDirection
    unit: str | None = None
    null_policy: str = "explicit_null"


@dataclass(frozen=True)
class MetricValue:
    run_id: str
    record_id: str | None
    estimator_name: str
    metric_name: str
    value: float | None
    stratum: Mapping[str, Any] = field(default_factory=dict)
    metadata: Mapping[str, Any] = field(default_factory=dict)


@dataclass(frozen=True)
class MetricBundle:
    per_series: tuple[MetricValue, ...]
    aggregate: tuple[MetricValue, ...]
    uncertainty: tuple[MetricValue, ...] = ()
    metadata: Mapping[str, Any] = field(default_factory=dict)


@dataclass(frozen=True)
class LeaderboardSpec:
    mode: BenchmarkMode
    component_metrics: tuple[str, ...]
    weights: Mapping[str, float]
    ranking_rule: str = "weighted_rank"
    tie_break_rule: str = "best_primary_metric"


@dataclass(frozen=True)
class LeaderboardRow:
    run_id: str
    estimator_name: str
    rank: int
    score: float
    component_values: Mapping[str, float | None]
    metadata: Mapping[str, Any] = field(default_factory=dict)
```

```python
@dataclass(frozen=True)
class ReportSpec:
    formats: tuple[str, ...]
    leaderboards: tuple[LeaderboardSpec, ...]
    figure_set: tuple[str, ...] = ()
    table_set: tuple[str, ...] = ()
    include_raw_exports: bool = True
    include_provenance: bool = True
    include_environment: bool = True
    export_root: str = "reports"
    naming_policy: str = "deterministic"
    compression_policy: str | None = None


@dataclass(frozen=True)
class ArtefactRecord:
    artefact_id: str
    run_id: str
    artefact_type: str
    format: str
    path: str
    hash: str | None = None
    created_at: str | None = None
    depends_on: tuple[str, ...] = ()


@dataclass(frozen=True)
class ReportBundle:
    run_id: str
    summary_table_path: str | None = None
    leaderboard_paths: tuple[str, ...] = ()
    figure_paths: tuple[str, ...] = ()
    latex_table_paths: tuple[str, ...] = ()
    html_report_path: str | None = None
    markdown_report_path: str | None = None
    manifest_copy_path: str | None = None
    result_store_path: str | None = None
    artefacts: tuple[ArtefactRecord, ...] = ()


@dataclass(frozen=True)
class BenchmarkManifest:
    manifest_id: str
    name: str
    mode: BenchmarkMode
    source_spec: Mapping[str, Any]
    contamination_spec: Mapping[str, Any] = field(default_factory=dict)
    segmentation_spec: Mapping[str, Any] = field(default_factory=dict)
    preprocessing_spec: Mapping[str, Any] = field(default_factory=dict)
    estimator_specs: tuple[EstimatorSpec, ...] = ()
    metric_specs: tuple[MetricSpec, ...] = ()
    leaderboard_specs: tuple[LeaderboardSpec, ...] = ()
    report_spec: ReportSpec | None = None
    execution_spec: Mapping[str, Any] = field(default_factory=dict)
```

```
    seed_spec: Mapping[str, Any] = field(default_factory=dict)
```

4.5 Generator and transformation interfaces

```
class BaseGenerator(ABC):
    @property
    @abstractmethod
    def family(self) -> str:
        ...

    @property
    @abstractmethod
    def version(self) -> str:
        ...

    @abstractmethod
    def generate(
        self,
        *,
        record_id: str,
        seed: int | None = None,
        metadata: Mapping[str, Any] | None = None,
    ) -> SeriesRecord:
        """
        Generate a new SeriesRecord.
        Must satisfy the generator contract:
        - values are non-empty
        - truth is present for synthetic sources
        - contamination history is initially empty
        - provenance is recorded
        """
        ...


class BaseContamination(ABC):
    @property
    @abstractmethod
    def name(self) -> str:
        ...

    @property
    @abstractmethod
    def family(self) -> str:
        ...

    @property
    @abstractmethod
    def version(self) -> str:
        ...

    @abstractmethod
    def apply(
        self,
```

```
        record: SeriesRecord,
        *,
        seed: int | None = None,
    ) -> SeriesRecord:
        """
        Return a transformed SeriesRecord whose provenance and
        contamination history are updated.
        """
        ...


class BasePreprocessor(ABC):
    @property
    @abstractmethod
    def name(self) -> str:
        ...

    @property
    @abstractmethod
    def version(self) -> str:
        ...

    @abstractmethod
    def apply(self, record: SeriesRecord) -> SeriesRecord:
        """
        Return a preprocessed SeriesRecord whose preprocessing history
        is updated without destroying provenance.
        """
        ...
```

4.6 Estimator interfaces

```
class BaseEstimator(ABC):
    @property
    @abstractmethod
    def spec(self) -> EstimatorSpec:
        """
        Must declare target estimand, assumptions, uncertainty support,
        and implementation metadata.
        """
        ...

    @abstractmethod
    def fit(self, record: SeriesRecord) -> EstimateResult:
        """
        Estimate the target quantity from a SeriesRecord and return
        a structured EstimateResult.
        Must never silently swallow failures.
        """
        ...


@runtime_checkable
```

```python
class SupportsBootstrapCI(Protocol):
    def bootstrap_ci(
        self,
        record: SeriesRecord,
        *,
        level: float = 0.95,
        n_boot: int = 1000,
        seed: int | None = None,
    ) -> tuple[float | None, float | None]:
        ...
```

4.7 Evaluation interfaces

```python
class BaseEvaluator(ABC):
    @abstractmethod
    def evaluate(
        self,
        manifest: BenchmarkManifest,
        records: Sequence[SeriesRecord],
        estimates: Sequence[EstimateResult],
    ) -> MetricBundle:
        """
        Compute benchmark metrics under the admissibility rules
        determined by the benchmark mode.
        """
        ...


class BaseLeaderboardBuilder(ABC):
    @abstractmethod
    def build(
        self,
        manifest: BenchmarkManifest,
        metrics: MetricBundle,
    ) -> tuple[LeaderboardRow, ...]:
        """
        Build leaderboard rows from aggregate metrics according to the
        declared LeaderboardSpec objects.
        """
        ...
```

4.8 Reporting and persistence interfaces

```python
class BaseReporter(ABC):
    @abstractmethod
    def build(
        self,
        manifest: BenchmarkManifest,
        metrics: MetricBundle,
        leaderboards: Sequence[LeaderboardRow],
        *,
```

```python
        report_spec: ReportSpec,
    ) -> ReportBundle:
        """
        Render human-readable and machine-readable artefacts and return
        a ReportBundle describing their locations.
        """
        ...


class BaseResultStore(ABC):
    @abstractmethod
    def write_run_metadata(
        self,
        manifest: BenchmarkManifest,
    ) -> None:
        ...

    @abstractmethod
    def write_records(
        self,
        records: Sequence[SeriesRecord],
    ) -> None:
        ...

    @abstractmethod
    def write_estimates(
        self,
        estimates: Sequence[EstimateResult],
    ) -> None:
        ...

    @abstractmethod
    def write_metrics(
        self,
        metrics: MetricBundle,
    ) -> None:
        ...

    @abstractmethod
    def write_leaderboards(
        self,
        rows: Sequence[LeaderboardRow],
    ) -> None:
        ...

    @abstractmethod
    def finalise(self) -> str:
        """
        Return the root path or persistent identifier for the result store.
        """
        ...
```

4.9 Registry interfaces

```
class BaseRegistry(ABC):
    @abstractmethod
    def register(self, name: str, obj: Any) -> None:
        ...

    @abstractmethod
    def get(self, name: str) -> Any:
        ...

    @abstractmethod
    def list(self) -> tuple[str, ...]:
        ...


class GeneratorRegistry(BaseRegistry):
    pass


class EstimatorRegistry(BaseRegistry):
    pass


class MetricRegistry(BaseRegistry):
    pass


class ReporterRegistry(BaseRegistry):
    pass
```

4.10 Benchmark orchestration interface

```
@dataclass(frozen=True)
class BenchmarkRunOutput:
    run_id: str
    records: tuple[SeriesRecord, ...]
    estimates: tuple[EstimateResult, ...]
    metrics: MetricBundle
    leaderboards: tuple[LeaderboardRow, ...]
    report_bundle: ReportBundle | None
    result_store_path: str | None = None


class BenchmarkRunner(ABC):
    @abstractmethod
    def run(
        self,
        manifest: BenchmarkManifest,
    ) -> BenchmarkRunOutput:
        """
        Execute a full benchmark run:
        1. construct or load records
```

```
        2. apply contaminations and preprocessing where relevant
        3. fit estimators
        4. evaluate metrics
        5. build leaderboards
        6. persist outputs
        7. generate reports
        """
        ...
```

4.11 Validation helpers

```
def validate_truth_compatibility(
    estimator_spec: EstimatorSpec,
    record: SeriesRecord,
) -> None:
    """
    Raise an error or warning if the estimator target estimand is not
    compatible with the record truth specification.
    """
    ...


def validate_metric_admissibility(
    metric_spec: MetricSpec,
    mode: BenchmarkMode,
    record: SeriesRecord,
) -> None:
    """
    Ensure that truth-based metrics are not computed in observational mode
    and that all metric preconditions are satisfied.
    """
    ...


def validate_manifest(manifest: BenchmarkManifest) -> None:
    """
    Perform structural and semantic checks on a benchmark manifest before
    execution begins.
    """
    ...
```

4.12 Recommended implementation pattern

The framework is best implemented using the following pattern:

PP1. use frozen `dataclass` objects for immutable schema records;

PP2. use abstract base classes for major pluggable components;

PP3. use small registries for component discovery and enrolment;

PP4. keep provenance and serialisation logic separate from numerical routines;

PP5. ensure that orchestration code is thin and declarative.

Remark 4.1. *This design deliberately avoids coupling mathematical logic, storage logic, and reporting logic. The separation is important because the framework is intended to function both as a research benchmark engine and as a long-term extensible platform for future estimator development.*

5 YAML Manifest Schema

This section specifies the YAML manifest schema used to declare reproducible benchmark runs. The manifest is the primary user-facing experiment specification and is intended to map directly onto the `BenchmarkManifest` object described in Section 4. The guiding principle is that a complete benchmark run should be reproducible from a single manifest file together with the corresponding software version and data sources.

5.1 Manifest design principles

The manifest schema is governed by the following principles.

YM1. Single-source specification. All benchmark-defining choices should be expressible in one manifest.

YM2. Mode awareness. Required and admissible fields depend on the benchmark mode.

YM3. Human readability. The manifest should remain readable and editable by researchers.

YM4. Machine validation. The manifest must admit structural and semantic validation before execution.

YM5. Deterministic execution. The manifest must support explicit random seed policy and reproducible export configuration.

5.2 Top-level schema

A valid YAML manifest shall contain the following top-level keys:

Key	Required	Purpose
`manifest_id`	yes	unique manifest identifier
`name`	yes	benchmark name
`description`	no	human-readable description
`mode`	yes	benchmark mode
`source`	yes	synthetic or observational source specification
`contamination`	mode-dependent	contamination specification
`segmentation`	no	windowing/segmentation rules
`preprocessing`	no	preprocessing pipeline
`estimators`	yes	enrolled estimator list
`metrics`	yes	metric declarations
`leaderboards`	no	leaderboard declarations
`report`	no	reporting/export configuration
`execution`	no	runtime and parallelisation settings
`seeds`	no	seed policy
`validation`	no	manifest validation policy

5.3 Mode field

The field

$$\texttt{mode} \in \{\texttt{ground_truth}, \texttt{stress_test}, \texttt{observational}\}$$

must be declared explicitly.

Remark 5.1. *There shall be no implicit default benchmark mode in any published or persisted run.*

5.4 Source schema

The `source` block describes the origin of the benchmark time series. Its required structure depends on the benchmark mode.

5.4.1 Ground-truth and stress-test source schema

For `ground_truth` and `stress_test` modes, the source should be specified as a generator grid:

```yaml
source:
  type: generator_grid
  generators:
    - family: fGn
      params:
        H: [0.3, 0.5, 0.7, 0.9]
        n: [1024, 4096]
        sigma: [1.0]
      replicates: 100

    - family: fBm
      params:
        H: [0.3, 0.5, 0.7, 0.9]
        n: [1024, 4096]
        sigma: [1.0]
      replicates: 100

    - family: ARFIMA
      params:
        p: [0]
        d: [0.0, 0.2, 0.4]
        q: [0]
        n: [1024, 4096]
      replicates: 100
```

Each generator entry must contain:

- `family`: generator family identifier;
- `params`: parameter grid;
- `replicates`: number of realisations per parameter combination.

5.4.2 Observational source schema

For `observational` mode, the source should specify a dataset loader:

```
source:
  type: dataset
  loader: csv
  paths:
    - data/eeg_subject01.csv
    - data/eeg_subject02.csv
  columns:
    value: signal
    time: time
  sampling_rate: 256.0
  annotations:
    subject_id_from_filename: true
    channel: Cz
```

Supported loader types may include:

- `csv`
- `api`
- `dataset_registry`
- `parquet`

5.5 Contamination schema

The `contamination` block is required in `stress_test` mode and must be absent or empty in `ground_truth` mode unless explicitly used for ablation studies.

A recommended structure is:

```
contamination:
  pairing: inherit_seed
  compose: false
  operators:
    - name: polynomial_trend
      family: trend
      params:
        order: [1, 2]
        strength: [0.1, 0.3]

    - name: outliers
      family: impulsive
      params:
        rate: [0.005, 0.01]
        amplitude: [5, 10]

    - name: heavy_tail_noise
      family: heavy_tail
      params:
        alpha: [1.2, 1.5, 1.8]
        scale: [0.1, 0.3]
```

Its fields are interpreted as follows:

- `pairing`: rule for pairing contaminated records with clean progenitors;
- `compose`: whether multiple operators are composed in the same scenario;
- `operators`: list of contamination operators and parameter grids.

Remark 5.2. *The default scientific design should preserve clean–contaminated pairing, since stress-test metrics such as drift and degradation ratios depend on paired provenance.*

5.6 Segmentation schema

The `segmentation` block is primarily relevant to observational mode, though it may also be used in synthetic experiments.

```
segmentation:
  enabled: true
  strategy: sliding_window
  window_length: 2048
  hop_length: 512
  min_length: 2048
  drop_incomplete: true
```

Admissible keys include:

- `enabled`
- `strategy`
- `window_length`
- `hop_length`
- `min_length`
- `drop_incomplete`

5.7 Preprocessing schema

The `preprocessing` block defines an ordered preprocessing pipeline.

```
preprocessing:
  steps:
    - name: demean
      params: {}

    - name: detrend
      params:
        method: linear

    - name: winsorise
      params:
        limits: [0.01, 0.99]
```

Each step must contain:

- `name`: preprocessing operator identifier;
- `params`: operator parameters.

Remark 5.3. *Preprocessing is part of benchmark provenance and must therefore be declared rather than silently embedded in loader logic.*

5.8 Estimator schema

The `estimators` block specifies the estimator set to be enrolled in the benchmark.

```
estimators:
  - name: DFA
    family: time_domain
    target_estimand: hurst_scaling_proxy
    params:
      min_scale: 8
      max_scale: 256

  - name: LocalWhittle
    family: spectral
    target_estimand: long_memory_parameter
    params:
      bandwidth: auto

  - name: WaveletLogVar
    family: wavelet
    target_estimand: hurst_self_similarity
    params:
      wavelet: db4
      j1: auto
      j2: auto
```

Each estimator entry should include:

- `name`
- `family`
- `target_estimand`
- `params`

Optional fields may include `version`, `supports_ci`, `assumptions`, and `notes`.

5.9 Metric schema

The `metrics` block declares which metrics are to be computed.

```
metrics:
  - name: bias
  - name: mae
  - name: rmse
```

```
  - name: validity_rate
  - name: runtime
  - name: coverage
    levels: [0.90, 0.95]
  - name: ci_width
    levels: [0.90, 0.95]
```

Mode-specific metrics may be declared similarly:

```
metrics:
  - name: relative_degradation_ratio
  - name: estimate_drift
  - name: coverage_collapse
  - name: instability
  - name: preprocessing_sensitivity
```

Remark 5.4. *Manifest validation must reject truth-based metrics in* `observational` *mode unless explicitly marked as inadmissible placeholders for internal use only.*

5.10 Leaderboard schema

The `leaderboards` block declares one or more ranking summaries.

```
leaderboards:
  - name: accuracy
    mode: ground_truth
    component_metrics: [mae, coverage_error, ci_width, validity_rate, runtime]
    weights:
      mae: 0.35
      coverage_error: 0.20
      ci_width: 0.10
      validity_rate: 0.20
      runtime: 0.15
    ranking_rule: weighted_rank
    tie_break_rule: mae

  - name: robustness
    mode: stress_test
    component_metrics: [relative_degradation_ratio, coverage_collapse, validity_collapse,
         runtime]
    weights:
      relative_degradation_ratio: 0.40
      coverage_collapse: 0.20
      validity_collapse: 0.20
      runtime: 0.20
    ranking_rule: weighted_rank
    tie_break_rule: relative_degradation_ratio
```

The validator must check that:

(L1) all component metrics are declared in the `metrics` block;

(L2) all weights are non-negative;

(L3) weights sum to unity within numerical tolerance;

(L4) the leaderboard mode is compatible with the manifest mode.

5.11 Report schema

The `report` block governs exports and rendering.

```
report:
  formats: [html, markdown, latex, csv, parquet, json]
  figure_set:
    - leaderboard_barplot
    - degradation_curve
    - calibration_plot
  table_set:
    - run_summary
    - estimator_metadata
    - per_stratum_metrics
    - leaderboard
    - failures
  include_raw_exports: true
  include_provenance: true
  include_environment: true
  export_root: reports
  naming_policy: deterministic
```

5.12 Execution schema

The `execution` block declares runtime behaviour.

```
execution:
  n_jobs: 8
  fail_fast: false
  cache_enabled: true
  overwrite_existing: false
  progress: true
```

Typical keys include:

- `n_jobs`
- `fail_fast`
- `cache_enabled`
- `overwrite_existing`
- `progress`

5.13 Seed schema

The `seeds` block controls reproducibility.

```
seeds:
  global_seed: 12345
  per_component:
    generators: true
```

```
    contaminations: true
    bootstrap: true
  pairing_policy: inherit_seed
```

Recommended keys are:

- `global_seed`
- `per_component`
- `pairing_policy`

Remark 5.5. *In stress-test mode, the seed policy should preserve clean–contaminated pairing wherever scientifically appropriate.*

5.14 Validation schema

The optional `validation` block specifies execution-time strictness.

```
validation:
  strict: true
  reject_unknown_keys: true
  require_truth_for_truth_metrics: true
  require_target_compatibility: true
```

5.15 Manifest validation rules

A manifest is valid if and only if the following conditions hold.

MV1. `manifest_id`, `name`, `mode`, `source`, `estimators`, and `metrics` are present.

MV2. `stress_test` mode requires a non-empty `contamination` block.

MV3. `ground_truth` mode must not declare contamination unless the experimental purpose explicitly permits it.

MV4. `observational` mode must not declare synthetic generator grids as its source.

MV5. every estimator must declare a `target_estimand`.

MV6. every requested metric must be admissible in the declared mode.

MV7. every leaderboard must refer only to declared metrics.

MV8. every leaderboard weight vector must sum to unity.

MV9. if `include_provenance` is true, then the manifest must be serialisable and persistable in full.

5.16 Canonical ground-truth manifest example

```yaml
manifest_id: gt_default_v1
name: canonical_ground_truth_suite
description: >
  Canonical synthetic benchmark for baseline estimator accuracy and calibration.
mode: ground_truth

source:
  type: generator_grid
  generators:
    - family: fGn
      params:
        H: [0.3, 0.5, 0.7, 0.9]
        n: [1024, 4096]
      replicates: 100

    - family: fBm
      params:
        H: [0.3, 0.5, 0.7, 0.9]
        n: [1024, 4096]
      replicates: 100

estimators:
  - name: DFA
    family: time_domain
    target_estimand: hurst_scaling_proxy
    params: {}

  - name: LocalWhittle
    family: spectral
    target_estimand: long_memory_parameter
    params: {}

metrics:
  - name: bias
  - name: mae
  - name: rmse
  - name: validity_rate
  - name: runtime
  - name: coverage
    levels: [0.95]

leaderboards:
  - name: accuracy
    mode: ground_truth
    component_metrics: [mae, coverage_error, validity_rate, runtime]
    weights:
      mae: 0.40
      coverage_error: 0.20
      validity_rate: 0.20
      runtime: 0.20
    ranking_rule: weighted_rank
    tie_break_rule: mae
```

```yaml
report:
  formats: [html, latex, csv, parquet, json]
  include_raw_exports: true
  include_provenance: true
  export_root: reports

execution:
  n_jobs: 8
  fail_fast: false

seeds:
  global_seed: 12345
```

5.17 Canonical stress-test manifest example

```yaml
manifest_id: st_default_v1
name: contamination_stress_suite
description: >
  Stress-test benchmark for estimator degradation under nonstationarity and heavy-tailed
      contamination.
mode: stress_test

source:
  type: generator_grid
  generators:
    - family: fGn
      params:
        H: [0.5, 0.7, 0.9]
        n: [4096]
      replicates: 100

contamination:
  pairing: inherit_seed
  compose: false
  operators:
    - name: polynomial_trend
      family: trend
      params:
        order: [1, 2]
        strength: [0.1, 0.3]

    - name: outliers
      family: impulsive
      params:
        rate: [0.005, 0.01]
        amplitude: [5, 10]

estimators:
  - name: DFA
    family: time_domain
    target_estimand: hurst_scaling_proxy
    params: {}
```

```
  - name: LocalWhittle
    family: spectral
    target_estimand: long_memory_parameter
    params: {}

metrics:
  - name: mae
  - name: validity_rate
  - name: runtime
  - name: relative_degradation_ratio
  - name: estimate_drift
  - name: validity_collapse
  - name: coverage_collapse

leaderboards:
  - name: robustness
    mode: stress_test
    component_metrics: [relative_degradation_ratio, validity_collapse, runtime]
    weights:
      relative_degradation_ratio: 0.50
      validity_collapse: 0.30
      runtime: 0.20
    ranking_rule: weighted_rank
    tie_break_rule: relative_degradation_ratio

report:
  formats: [html, latex, csv, parquet, json]
  figure_set: [degradation_curve]
  export_root: reports

execution:
  n_jobs: 8

seeds:
  global_seed: 12345
  pairing_policy: inherit_seed
```

5.18 Canonical observational manifest example

```
manifest_id: obs_default_v1
name: biomedical_observational_suite
description: >
  Observational benchmark for estimator stability and sensitivity on biomedical time
      series.
mode: observational

source:
  type: dataset
  loader: csv
  paths:
    - data/eeg_subject01.csv
    - data/eeg_subject02.csv
```

```yaml
  columns:
    value: signal
    time: time
  sampling_rate: 256.0

segmentation:
  enabled: true
  strategy: sliding_window
  window_length: 2048
  hop_length: 512
  drop_incomplete: true

preprocessing:
  steps:
    - name: demean
      params: {}
    - name: detrend
      params:
        method: linear

estimators:
  - name: DFA
    family: time_domain
    target_estimand: hurst_scaling_proxy
    params: {}

  - name: WaveletLogVar
    family: wavelet
    target_estimand: hurst_self_similarity
    params:
      wavelet: db4

metrics:
  - name: instability
  - name: preprocessing_sensitivity
  - name: runtime
  - name: validity_rate

leaderboards:
  - name: stability
    mode: observational
    component_metrics: [instability, preprocessing_sensitivity, validity_rate, runtime]
    weights:
      instability: 0.40
      preprocessing_sensitivity: 0.25
      validity_rate: 0.20
      runtime: 0.15
    ranking_rule: weighted_rank
    tie_break_rule: instability

report:
  formats: [html, csv, parquet, json]
  include_provenance: true
  export_root: reports
```

```
execution:
  n_jobs: 4

seeds:
  global_seed: 12345
```

5.19 Recommended parser behaviour

A YAML manifest parser should implement the following sequence:

YP1. load the YAML document;

YP2. validate top-level structure;

YP3. validate mode-specific admissibility rules;

YP4. construct strongly typed internal objects;

YP5. resolve registry references for generators, estimators, metrics, and reporters;

YP6. perform semantic checks on weights, target compatibility, and metric admissibility;

YP7. produce a fully instantiated `BenchmarkManifest`.

Remark 5.6. *The manifest parser should fail early and explicitly on structural or semantic violations. Silent coercion is inappropriate for a research benchmark framework.*

6 Development Roadmap and Package Build Strategy

This section specifies the development roadmap and package build strategy for the benchmarking framework. The objective is to translate the formal architecture into an implementation plan that is incremental, reproducible, and scientifically disciplined. The roadmap is designed so that each phase produces a usable and testable research artefact, rather than deferring all value to a final monolithic release.

6.1 Development philosophy

The implementation strategy is guided by the following principles.

DR1. Specification before expansion. Core schema, contracts, and manifest behaviour should be stabilised before adding many estimators or contaminations.

DR2. Vertical slices over horizontal accumulation. Each phase should implement an end-to-end workflow, even if initially narrow in scope.

DR3. Scientific validation at each phase. New components should not merely compile; they should satisfy benchmark-specific validation and reproducibility checks.

DR4. Minimal stable core, optional rich extras. The base package should remain lean, while heavy reporting, documentation, and dataset integrations may be exposed through optional dependency groups.

DR5. Reproducibility as a release criterion. A feature is not complete unless its outputs can be serialised, audited, and regenerated from a manifest.

6.2 Recommended package layout

The package should follow a `src/`-layout with a modern build backend and a single `pyproject.toml` entry point. The recommended high-level layout is:

```
project_root/
  pyproject.toml
  README.md
  LICENSE
  CITATION.cff
  CHANGELOG.md
  RESEARCH_USAGE.md
  src/
    lrdbench/
      ...
  tests/
  docs/
  configs/
  examples/
  scripts/
```

Remark 6.1. *A `src/`-layout reduces import ambiguity during development and improves packaging discipline.*

6.3 Build strategy

The package build strategy should satisfy four goals:

BS1. support local editable development installs;

BS2. support source distributions and wheels;

BS3. support optional extras for development, documentation, and reporting;

BS4. support deterministic CI builds.

A recommended `pyproject.toml` design is:

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "lrdbench"
version = "0.1.0"
description = "Benchmarking framework for long-range dependence estimators"
readme = "README.md"
requires-python = ">=3.11"
license = {text = "MIT"}
authors = [
  {name = "Author Name", email = "author@example.com"}
]
dependencies = [
  "numpy",
  "scipy",
```

```
  "pandas",
  "pyyaml",
]

[project.optional-dependencies]
dev = [
  "pytest",
  "pytest-cov",
  "mypy",
  "ruff",
]
docs = [
  "mkdocs",
  "mkdocs-material",
]
reports = [
  "matplotlib",
  "jinja2",
  "tabulate",
]
all = [
  "lrdbench[dev,docs,reports]"
]

[project.scripts]
lrdbench = "lrdbench.cli.main:main"
```

Remark 6.2. *The precise build backend is not the scientific issue; what matters is that the build process is explicit, lightweight, and compatible with reproducible CI and release workflows.*

6.4 Dependency strategy

Dependencies should be organised into layers.

Core dependencies. These should be as small as possible and include only what is necessary for:

- numerical computation,
- manifest parsing,
- structured data handling,
- core serialisation.

Optional reporting dependencies. Plotting, HTML report rendering, and publication-oriented exports should be optional.

Optional documentation dependencies. Documentation tooling should not be required for package installation.

Optional data integration dependencies. Biomedical loaders, API clients, or connector-specific interfaces should be placed behind optional extras if they are introduced later.

Remark 6.3. *A lean core installation matters because benchmark users may wish to run large numerical studies on minimal environments where documentation and rendering dependencies are irrelevant.*

6.5 Development phases

The framework should be developed in phases. Each phase should produce a scientifically meaningful milestone.

Phase 0: specification freeze

Goal: freeze the conceptual spine before large-scale implementation.
Deliverables:

(P0.1) core schema and contracts;

(P0.2) metric schema and evaluation protocol;

(P0.3) reporting schema and artefact protocol;

(P0.4) Python-facing interfaces;

(P0.5) YAML manifest schema.

Exit criterion: the design documents are internally consistent and sufficient to guide implementation without conceptual ambiguity.

Phase 1: minimal vertical slice

Goal: implement the smallest end-to-end benchmark that is scientifically coherent.
Scope:

- generators: `fGn`, `fBm`, `ARFIMA(0,d,0)`;
- estimators: a small baseline set;
- metrics: bias, MAE, RMSE, validity rate, runtime;
- reporting: CSV + HTML summary + manifest export;
- orchestration: one working `BenchmarkRunner`.

Exit criterion: a ground-truth benchmark can be declared in YAML, executed from the CLI, and exported reproducibly.

Phase 2: uncertainty and calibration

Goal: elevate the framework from point-estimate benchmarking to uncertainty-aware benchmarking.
Scope:

- confidence interval handling;
- empirical coverage computation;
- interval width summaries;
- missing-uncertainty handling policy;
- coverage reporting in HTML/CSV/LaTeX outputs.

Exit criterion: the framework supports uncertainty-aware benchmark reports on ground-truth datasets.

Phase 3: stress-test mode

Goal: implement contamination-based robustness benchmarking.
Scope:

- contamination operators: outliers, level shifts, polynomial trends, heavy-tail noise;
- paired clean–contaminated provenance;
- drift and degradation metrics;
- robustness leaderboards;
- degradation plots.

Exit criterion: contamination studies are reproducible from manifests and yield paired clean–contaminated reports.

Phase 4: richer generators and extensibility

Goal: broaden the framework beyond the minimal canonical core.
Scope:

- add `MRW` and `fOU`;
- improve registry design;
- add custom-estimator enrolment examples;
- publish contribution templates for new estimators.

Exit criterion: external contributors can add new estimators or generators without modifying the internal orchestration logic.

Phase 5: observational mode

Goal: support biomedical and custom real-world data without misusing truth-based metrics.

Scope:

- CSV/API dataset loaders;
- segmentation and preprocessing pipelines;
- instability and sensitivity metrics;
- observational reports and stability leaderboards.

Exit criterion: observational datasets can be benchmarked with mode-appropriate metrics and reports.

Phase 6: hardening and public release

Goal: stabilise the framework for broader public or collaborative research use.

Scope:

- regression-test baselines;
- documentation site;
- example manifests;
- release automation;
- contribution and governance documents.

Exit criterion: reproducible tagged release with documentation, examples, and CI-backed package builds.

6.6 Milestone-oriented roadmap

A practical milestone schedule is:

Milestone	Focus	Output
M1	Phase 1	minimal ground-truth benchmark package
M2	Phase 2	UQ-aware baseline benchmark
M3	Phase 3	contamination stress-test benchmark
M4	Phase 4	extensible plugin-ready benchmark engine
M5	Phase 5	observational/biomedical benchmark mode
M6	Phase 6	public research release

6.7 Testing strategy within the roadmap

Testing should evolve in parallel with functionality.

Unit tests. These should verify:

- schema construction;
- manifest parsing;
- registry behaviour;
- output serialisation;
- failure handling;
- report-path generation.

Integration tests. These should verify:

- end-to-end benchmark execution from manifest;
- consistency between result-store exports and reports;
- pairing integrity in stress-test mode.

Statistical tests. These should verify:

- generator sanity properties;
- broad estimator behaviour on canonical data;
- stability of summary metrics under fixed seeds.

Regression tests. These should preserve benchmark outputs for selected smoke manifests and detect unexpected scientific drift.

6.8 Continuous integration strategy

The CI pipeline should be layered.

CI1. Static checks: formatting, linting, and type checking.

CI2. Fast tests: unit and manifest-validation tests.

CI3. Integration smoke tests: one small benchmark per mode where implemented.

CI4. Documentation checks: build docs and example manifests.

CI5. Release checks: build wheel and source distribution on tags.

A typical repository workflow layout is:

```
.github/workflows/
  lint.yml
  tests.yml
  smoke-benchmark.yml
  docs.yml
  release.yml
```

6.9 Versioning and release strategy

The package should use semantic versioning or an equivalent explicit release policy.

Recommended interpretation.

- `0.x.y`: active design and API stabilisation period;
- `1.0.0`: first stable research release with frozen core interfaces;
- patch releases: bug fixes and non-breaking internal improvements;
- minor releases: additive features;
- major releases: interface or schema breaking changes.

Release artefacts. Each tagged release should include:

- source distribution;
- wheel;
- release notes;
- manifest examples;
- benchmark smoke outputs or checksums where appropriate.

6.10 Branching and contribution model

A simple branching strategy is sufficient.

- `main`: stable branch;
- short-lived feature branches for implementation units;
- pull requests gated by CI and code review;
- tagged releases from `main`.

Contributions should be categorised by type:

(C1) generators;

(C2) contaminations;

(C3) estimators;

(C4) metrics;

(C5) reporting templates;

(C6) documentation and examples.

Remark 6.4. *Estimator contributions are scientifically meaningful contributions, not merely software plugins. Their metadata and assumptions therefore require stricter review than cosmetic or infrastructural changes.*

6.11 Coding standards

The codebase should follow the following standards.

CS1. type-annotated public interfaces;

CS2. immutable schema records where feasible;

CS3. explicit exceptions rather than silent coercion;

CS4. pure functions or side-effect-minimised methods for numerical logic where practical;

CS5. deterministic serialisation of manifests and outputs;

CS6. clear distinction between scientific logic and infrastructure logic.

6.12 Documentation strategy within the roadmap

Documentation should grow with the phases.

Phase 1 documentation.

- installation;
- quickstart;
- first ground-truth benchmark example.

Phase 3 documentation.

- contamination manifests;
- robustness metric interpretation;
- paired clean–contaminated design notes.

Phase 5 documentation.

- observational mode safeguards;
- segmentation and preprocessing examples;
- mode-appropriate interpretation guidance.

6.13 Minimal implementation backlog

The first concrete implementation backlog should consist of the following tasks:

B1. initialise package skeleton and build configuration;

B2. implement immutable core schema records;

B3. implement manifest parser and validator;

B4. implement one generator registry and one estimator registry;

B5. implement baseline generators (`fGn`, `fBm`, `ARFIMA(0,d,0)`);

B6. implement one minimal estimator set;

B7. implement ground-truth evaluator;

B8. implement minimal result store;

B9. implement HTML/CSV report builder;

B10. implement CLI command for manifest execution.

Remark 6.5. *This backlog is intentionally narrow. The objective is to produce a functioning benchmark engine early, then widen the scientific surface area in controlled steps.*

6.14 Definition of done

A feature or phase should be considered complete only if:

DoD1. it is covered by automated tests;

DoD2. it is documented;

DoD3. it is manifest-addressable where applicable;

DoD4. it exports provenance-complete results;

DoD5. it does not violate mode-specific admissibility rules;

DoD6. it can be reproduced from a clean environment.

Remark 6.6. *For this framework, "done" does not simply mean "implemented". It means "implemented, validated, documented, and reproducible".*